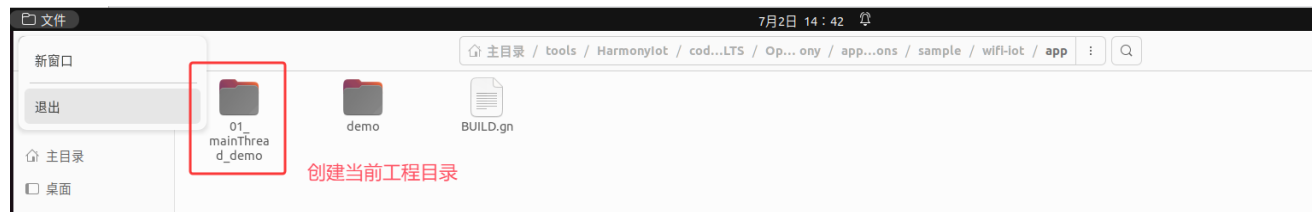


# 知识点1 工程的配置和编译

## 1. 进入到工程目录路径中：

/home/edu/tools/Harmonyot/code-v3.0-LTS/OpenHarmony/applications/sample/wifi-iot/app

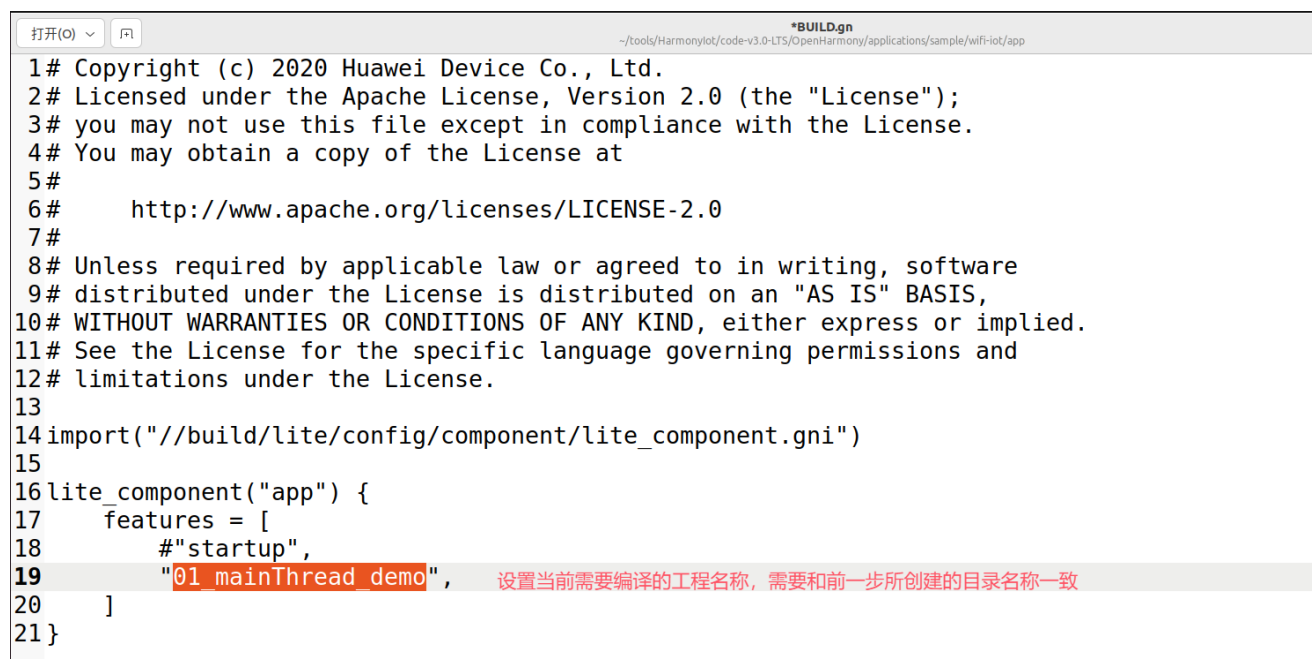
## 2. 创建新的目录作为工程目录：



注意：当前工程名称即为：**01\_mainThread\_demo**

## 3. 修改app中的编译规则文件：BUILD.gn

编译目录的配置选择



## 4. 进入到工程目录01\_mainThread\_demo中

1) 添加源程序文件或者头文件，入口程序源码：**mainThread\_demo.c**

```
1  #include <stdio.h>
2  #include <unistd.h>
3  #include "ohos_init.h"
4  #include "cmsis_os2.h"
5  #include "hi_time.h"
6  #include "hi_io.h"
7  #include "hi_gpio.h"
8
9  /* 定义子线程执行函数 */
10 void thread_func01(void *arg)
11 {
12     while(1)
```

```

13     {
14         printf("[ThreadTestTask]\n");
15         osDelay(300); //延时3s
16     }
17 }
18 /* 自定义线程入口函数 */
19 static void ThreadMainTask(void)
20 {
21     //定义线程属性
22     osThreadAttr_t attr;
23     attr.name = "thread_func01";          /* 所创建子线程的线程名称 */
24     attr.stack_size = 10*1024;           /* 设置子线程栈空间大小 */
25     attr.priority = osPriorityNormal;
26     if (osThreadNew((osThreadFunc_t)thread_func01, NULL, &attr) == NULL)
27     {
28         printf("[ThreadTestTask] Failed to create Task01!\n");
29     }
30 }
31 /* 声明函数入口 */
32 APP_FEATURE_INIT(ThreadMainTask);

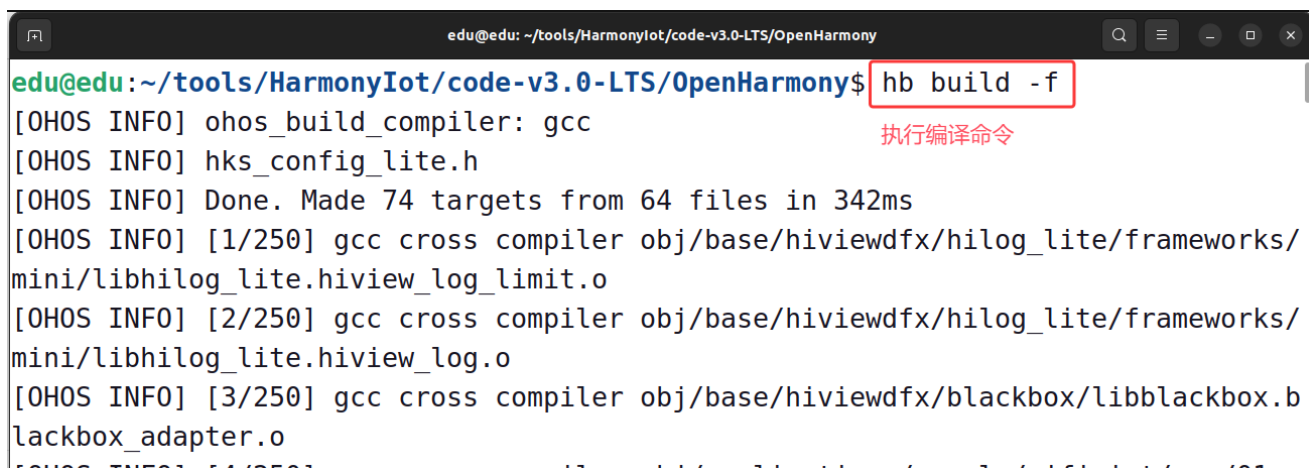
```

## 2) 配置编译规则，配置工程目录中的编译规则文件：BUILD.gn



## 5. 编译工程

在源码目录中执行编译命令:hb build -f



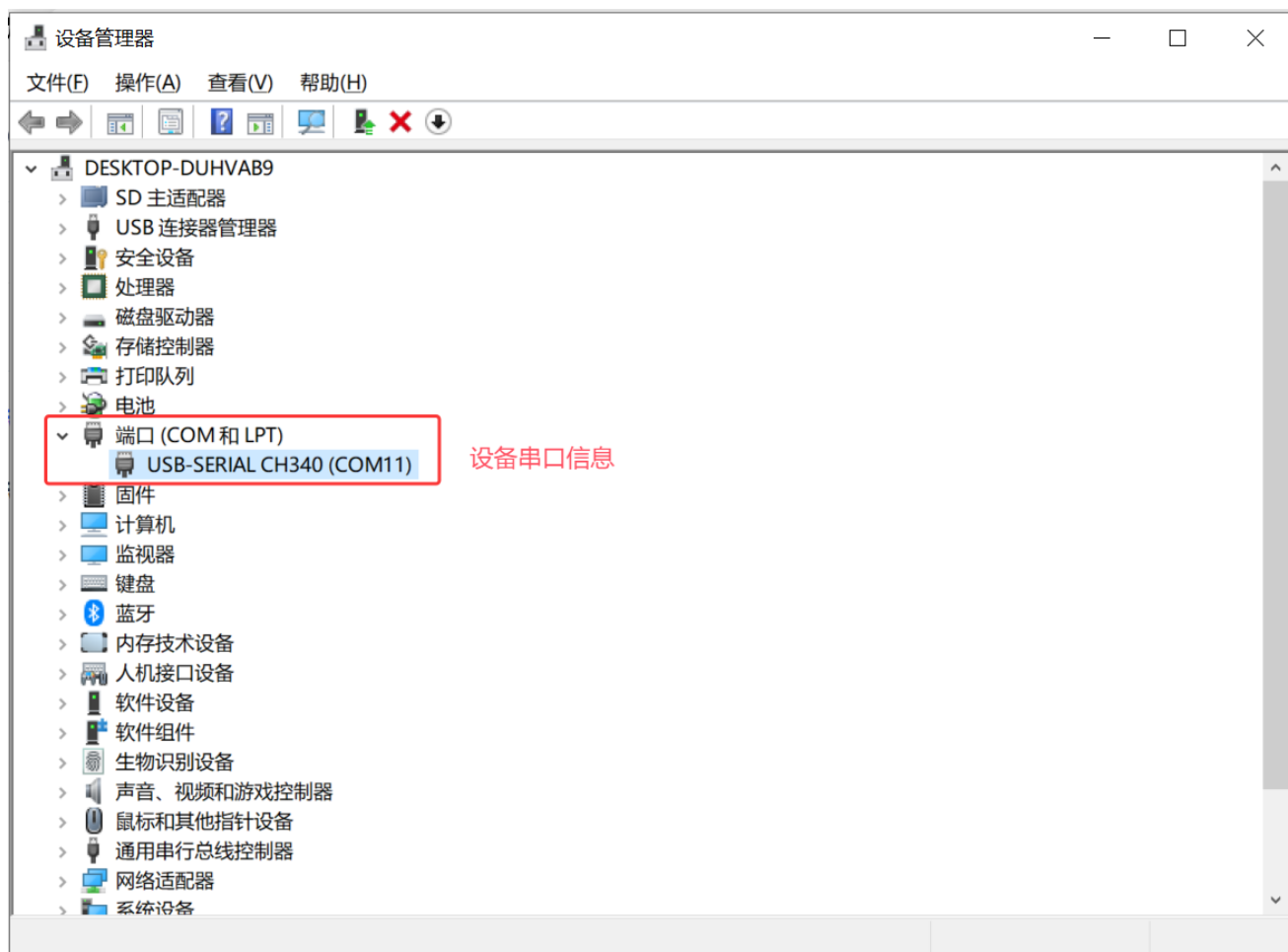
注意:

```
[OHOS INFO] /home/edu/tools/HarmonyIot/code-v3.0-LTS/OpenHarmony/vendor/hisilicon/hispark_pegasus/fs.yml not found, stop packing fs. If the product does not need to be packaged, ignore it.
[OHOS INFO] wifiiot_hispark_pegasus build success
[OHOS INFO] cost time: 0:00:29
edu@edu:~/tools/HarmonyIot/code-v3.0-LTS/OpenHarmony$
```

说明编译成功

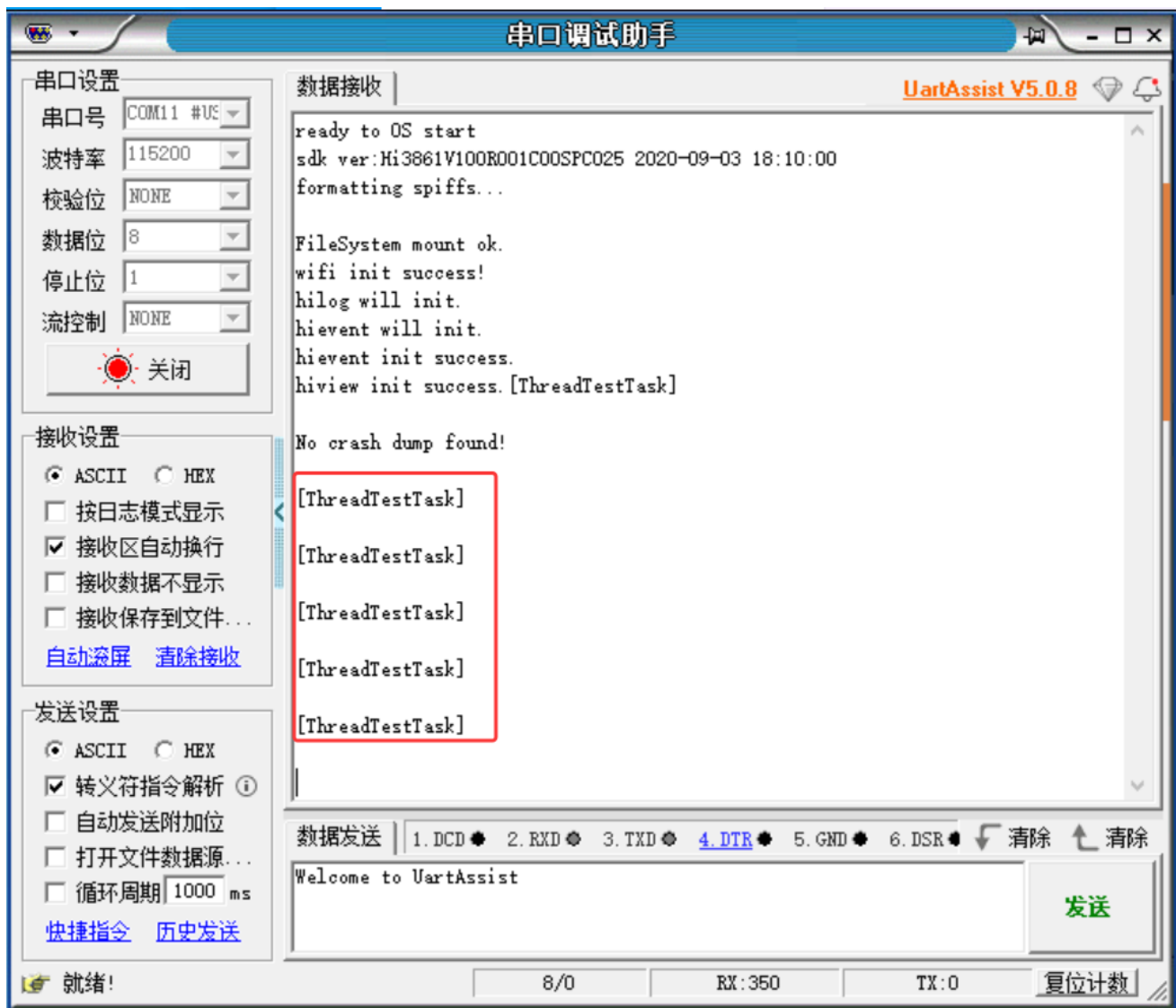
## 知识点2 程序的烧写

1. 将所生成的可执行程序文件 `Hi3861_wifiiot_app_allinone.bin` 拷贝到PC机中。
2. 将设备模块和PC机进行连接。
3. 在设备管理器中查看设备端口



4. 启动烧写工具完成镜像的烧写





## 知识点3 GPIO编程

### 1 GPIO相关的API接口

#### 1. 初始化gpio模块函数

```
1 hi_u32 hi_gpio_init(hi_void);
```

#### 2. 设置gpio管脚输入/输出方向函数

```
1 hi_u32 hi_gpio_set_dir(hi_gpio_idx id, hi_gpio_dir dir);
```

2 参数:

3 参数1: id是需要设置gpio引脚编号

```
4 typedef enum {
```

```
5     HI_GPIO_IDX_0,    /**< GPIO0*/
```

```
6     HI_GPIO_IDX_1,    /**< GPIO1*/
```

```

7         HI_GPIO_IDX_2,      /**< GPIO2*/
8         HI_GPIO_IDX_3,      /**< GPIO3*/
9         HI_GPIO_IDX_4,      /**< GPIO4*/
10        HI_GPIO_IDX_5,      /**< GPIO5*/
11        HI_GPIO_IDX_6,      /**< GPIO6*/
12        HI_GPIO_IDX_7,      /**< GPIO7*/
13        HI_GPIO_IDX_8,      /**< GPIO8*/
14        HI_GPIO_IDX_9,      /**< GPIO9*/
15        HI_GPIO_IDX_10,     /**< GPIO10*/
16        HI_GPIO_IDX_11,     /**< GPIO11*/
17        HI_GPIO_IDX_12,     /**< GPIO12*/
18        HI_GPIO_IDX_13,     /**< GPIO13*/
19        HI_GPIO_IDX_14,     /**< GPIO14*/
20        HI_GPIO_IDX_MAX,    /**< Maximum value, which cannot be used.CNcomment:最大
值，不可输入使用CNend*/
21    } hi_gpio_idx;
22    参数2: dir表示gpio输入/输出方向
23    typedef enum {
24        HI_GPIO_DIR_IN = 0,    /**< Input.CNcomment:输入方向CNend*/
25        HI_GPIO_DIR_OUT       /**< Output.CNcomment:输出方向CNend*/
26    } hi_gpio_dir;

```

### 3. 设置管脚输出高低电平状态

```

1 hi_u32 hi_gpio_set_output_val(hi_gpio_idx id, hi_gpio_value val);
2 参数:
3     参数1: id是需要设置gpio引脚编号
4     参数2: val是设置的输出电平状态值
5     typedef enum {
6         HI_GPIO_VALUE0 = 0,    /**< Low level.CNcomment:低电平CNend*/
7         HI_GPIO_VALUE1       /**< High level.CNcomment:高电平CNend*/
8     } hi_gpio_value;

```

### 4. 配置管脚的复用功能

因为hi3861芯片管脚数量有限，为了能够支持更多的外设类型，引脚提供复用功能引脚。

```

1 hi_u32 hi_io_set_func(hi_io_name id, hi_u8 val);
2 参数:
3     参数1: id是管脚编号

```

```

4      typedef enum {
5          HI_IO_NAME_GPIO_0,      /**< GPIO0 */
6          HI_IO_NAME_GPIO_1,      /**< GPIO1 */
7          HI_IO_NAME_GPIO_2,      /**< GPIO2 */
8          HI_IO_NAME_GPIO_3,      /**< GPIO3 */
9          HI_IO_NAME_GPIO_4,      /**< GPIO4 */
10         HI_IO_NAME_GPIO_5,      /**< GPIO5 */
11         HI_IO_NAME_GPIO_6,      /**< GPIO6 */
12         HI_IO_NAME_GPIO_7,      /**< GPIO7 */
13         HI_IO_NAME_GPIO_8,      /**< GPIO8 */
14         HI_IO_NAME_GPIO_9,      /**< GPIO9 */
15         HI_IO_NAME_GPIO_10,     /**< GPIO10 */
16         HI_IO_NAME_GPIO_11,     /**< GPIO11 */
17         HI_IO_NAME_GPIO_12,     /**< GPIO12 */
18         HI_IO_NAME_GPIO_13,     /**< GPIO13 */
19         HI_IO_NAME_GPIO_14,     /**< GPIO14 */
20         HI_IO_NAME_SFC_CSN,     /**< SFC_CSN */
21         HI_IO_NAME_SFC_I01,     /**< SFC_I01 */
22         HI_IO_NAME_SFC_I02,     /**< SFC_I02 */
23         HI_IO_NAME_SFC_I00,     /**< SFC_I00 */
24         HI_IO_NAME_SFC_CLK,     /**< SFC_CLK */
25         HI_IO_NAME_SFC_I03,     /**< SFC_I03 */
26         HI_IO_NAME_MAX,
27     } hi_io_name;
28     参数2: val值设置复用功能
29     typedef enum {
30         HI_IO_FUNC_GPIO_2_GPIO,      /* 通用输入输出管脚功能 */
31         HI_IO_FUNC_GPIO_2_UART1_RTS_N = 2, /* USART1复用功能 */
32         HI_IO_FUNC_GPIO_2_SPI1_TXD,
33         HI_IO_FUNC_GPIO_2_JTAG_TRSTN,
34         HI_IO_FUNC_GPIO_2_PWM2_OUT,
35         HI_IO_FUNC_GPIO_2_SSI_CLK = 7,
36     } hi_io_func_gpio_2;

```

## 5. 设置引脚上拉和下拉的状态

```

1  hi_u32 hi_io_set_pull(hi_io_name id, hi_io_pull val);
2  参数:
3      id:GPIO引脚号
4      val:上拉或下拉

```

```

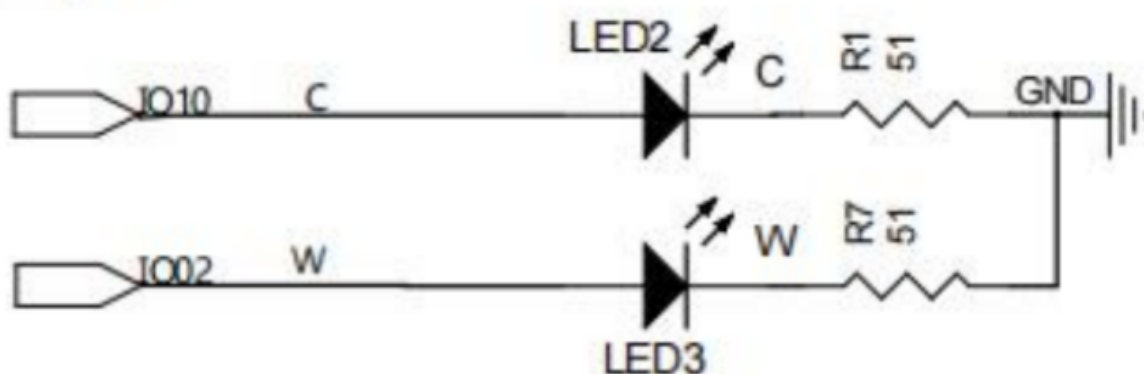
5         typedef enum {
6             HI_IO_PULL_NONE,    /**<无上下拉*/
7             HI_IO_PULL_UP,      /**<上拉*/
8             HI_IO_PULL_DOWN,    /**<下拉*/
9             HI_IO_PULL_MAX,     /**<无效值*/
10        } hi_io_pull;

```

6.

## 2 LED编程

### 1. LED原理图



LED电子设备为发光二极管，工作原理是存在正向电位差的时候导通点亮，否则熄灭。

引脚编号：IO02

需要将引脚设置为输出模式，电位状态 1 = 点亮；0 = 熄灭。

### 2. LED控制编程实现

- 1) 完成工程文件的相关配置
- 2) LED控制源程序的编程：led\_demo.c

```

1  #include <stdio.h>
2  #include <unistd.h>
3  #include "ohos_init.h"
4  #include "cmsis_os2.h"
5  #include "hi_time.h"
6  #include "hi_io.h"
7  #include "hi_gpio.h"
8
9  #define LED_PIN          HI_IO_NAME_GPIO_2 // for hispark_pegasus
10 #define LED_FUN_GPIO     HI_IO_FUNC_GPIO_2_GPIO
11
12 void LED_Init(void)

```



```

13 {
14     /* gpio模块 初始化 */
15     hi_gpio_init();
16
17     /* GPIO引脚设置为通用GPIO引脚功能 */
18     hi_io_set_func(LED_PIN, LED_FUN_GPIO);
19
20     /* 设置GPIO管脚为输出 */
21     hi_gpio_set_dir(LED_PIN, HI_GPIO_DIR_OUT);
22 }
23
24 void led_thread_func(void *arg)
25 {
26     LED_Init();
27
28     while(1) {
29         hi_gpio_set_output_val(LED_PIN, HI_GPIO_VALUE0); /* 引脚状态设置为低电平 */
30         osDelay(100); //延时1s
31         hi_gpio_set_output_val(LED_PIN, HI_GPIO_VALUE1); /* 引脚状态设置为高电平 */
32         osDelay(100); //延时3s
33     }
34 }
35
36 static void ThreadLedTask(void)
37 {
38     //定义线程属性
39     osThreadAttr_t attr;
40     attr.name = "led_thread_func";
41     attr.stack_size = 10*1024;
42     attr.priority = osPriorityNormal;
43     if (osThreadNew((osThreadFunc_t)led_thread_func, NULL, &attr) == NULL)
44     {
45         printf("[ThreadTestTask] Falied to create Task01!\n");
46     }
47 }
48 /* 声明函数入口 */
49 APP_FEATURE_INIT(ThreadLedTask);

```

## 3 RGB\_LED编程

### 1. RGB外设引脚

引脚连接:

RED\_LED: GPIO13

GREEN\_LED: GPIO12

BLUE\_LED: GPIO9

## 2. RGB\_LED编程

1) 完成工程目录的创建和文件的配置

2) 实现RGB\_LED编程: rgb\_demo.c

```
1  #include <stdio.h>
2  #include <unistd.h>
3  #include "ohos_init.h"
4  #include "cmsis_os2.h"
5  #include "hi_time.h"
6  #include "hi_io.h"
7  #include "hi_gpio.h"
8
9  #define RED_PIN                HI_IO_NAME_GPIO_13 // for hispark_pegasus
10 #define RED_FUN_GPIO           HI_IO_FUNC_GPIO_2_GPIO
11 #define GREEN_PIN              HI_IO_NAME_GPIO_12 // for hispark_pegasus
12 #define GREEN_FUN_GPIO         HI_IO_FUNC_GPIO_2_GPIO
13 #define BLUE_PIN               HI_IO_NAME_GPIO_9  // for hispark_pegasus
14 #define BLUE_FUN_GPIO          HI_IO_FUNC_GPIO_2_GPIO
15
16 void RGB_Init(void)
17 {
18     /* gpio 初始化 */
19     hi_gpio_init();
20
21     /* RED GPIO引脚设置为通用GPIO引脚功能 */
22     hi_io_set_func(RED_PIN, RED_FUN_GPIO);
23     /* 设置GPIO管脚为输出 */
24     hi_gpio_set_dir(RED_PIN, HI_GPIO_DIR_OUT);
25
26     /* GREEN GPIO引脚设置为通用GPIO引脚功能 */
27     hi_io_set_func(GREEN_PIN, GREEN_FUN_GPIO);
28     /* 设置GPIO管脚为输出 */
29     hi_gpio_set_dir(GREEN_PIN, HI_GPIO_DIR_OUT);
30
31     /* BLUE GPIO引脚设置为通用GPIO引脚功能 */
32     hi_io_set_func(BLUE_PIN, BLUE_FUN_GPIO);
33     /* 设置GPIO管脚为输出 */
```

```

34     hi_gpio_set_dir(BLUE_PIN, HI_GPIO_DIR_OUT);
35 }
36
37 void rgb_thread_func(void *arg)
38 {
39     RGB_Init();
40
41     while(1)
42     {
43         hi_gpio_set_ouput_val(RED_PIN, HI_GPIO_VALUE0); /* 引脚状态设置为低电平 */
44         osDelay(100); //延时1s
45         hi_gpio_set_ouput_val(RED_PIN, HI_GPIO_VALUE1); /* 引脚状态设置为高电平 */
46         osDelay(100); //延时1s
47         hi_gpio_set_ouput_val(GREEN_PIN, HI_GPIO_VALUE0); /* 引脚状态设置为低电平 */
48         osDelay(100); //延时1s
49         hi_gpio_set_ouput_val(GREEN_PIN, HI_GPIO_VALUE1); /* 引脚状态设置为高电平 */
50         osDelay(100); //延时1s
51         hi_gpio_set_ouput_val(BLUE_PIN, HI_GPIO_VALUE0); /* 引脚状态设置为低电平 */
52         osDelay(100); //延时1s
53         hi_gpio_set_ouput_val(BLUE_PIN, HI_GPIO_VALUE1); /* 引脚状态设置为高电平 */
54         osDelay(100); //延时1s
55     }
56 }
57
58 static void ThreadRGBTask(void)
59 {
60     //定义线程属性
61     osThreadAttr_t attr;
62     attr.name = "rgb_thread_func";
63     attr.stack_size = 10*1024;
64     attr.priority = osPriorityNormal;
65     if (osThreadNew((osThreadFunc_t)rgb_thread_func, NULL, &attr) == NULL)
66     {
67         printf("[ThreadTestTask] Falied to create Task01!\n");
68     }
69 }

```

## 4 蜂鸣器编程

### 4.1 蜂鸣器分类

蜂鸣器根据内部是否包含晶振分为有源蜂鸣器和无源蜂鸣器两种类型

#### 1. 有源蜂鸣器

内部包含晶振，只需要在上电的时候，晶振自动震荡发声，否则不会发声。

#### 2. 无源蜂鸣器

内部不包含震荡的晶振，需要外部时钟提供震荡频率，一般使用PWM波实现对蜂鸣器的控制。

## 4.2 硬件的连接

蜂鸣器设备：

VCC ---> 3v3

GND ---> GND

I/O ---> IO14

## 4.3 程序的设计

#### 1. 创建工程目录及其文件，并完成配置

#### 2. 实现蜂鸣器编程：

```
1  #include <stdio.h>
2  #include <unistd.h>
3  #include "ohos_init.h"
4  #include "cmsis_os2.h"
5  #include "hi_time.h"
6  #include "hi_io.h"
7  #include "hi_gpio.h"
8
9  #define BEEP_PIN          HI_IO_NAME_GPIO_7 // for hispark_pegasus
10 #define BEEP_FUN_GPIO     HI_IO_FUNC_GPIO_2_GPIO
11
12 void BEEP_Init(void)
13 {
14     /* gpio 初始化 */
15     hi_gpio_init();
16
17     /* GPIO引脚设置为通用GPIO引脚功能 */
18     hi_io_set_func(BEEP_PIN, BEEP_FUN_GPIO);
19
20     /* 设置GPIO管脚为输出 */
21     hi_gpio_set_dir(BEEP_PIN, HI_GPIO_DIR_OUT);
22 }
23
24 void beep_thread_func(void *arg)
```

```

25 {
26     BEEP_Init();
27
28     while(1)
29     {
30         hi_gpio_set_ouput_val(BEEP_PIN, HI_GPIO_VALUE0);          /* 引脚状态设置为低电平 */
31         osDelay(500); //延时5s
32         hi_gpio_set_ouput_val(BEEP_PIN, HI_GPIO_VALUE1);          /* 引脚状态设置为高电平 */
33         osDelay(500); //延时5s
34     }
35 }
36
37 static void ThreadBeepTask(void)
38 {
39     //定义线程属性
40     osThreadAttr_t attr;
41     attr.name = "beep_thread_func";
42     attr.stack_size = 10*1024;
43     attr.priority = osPriorityNormal;
44     if (osThreadNew((osThreadFunc_t)beep_thread_func, NULL, &attr) == NULL)
45     {
46         printf("[ThreadTestTask] Falied to create Task01!\n");
47     }
48 }
49 /* 声明函数入口 */
50 APP_FEATURE_INIT(ThreadBeepTask);

```

## 5 舵机编程

### 5.1 SG90舵机概述

SG90是一种常见的微型伺服电机，广泛用于远程控制汽车、船只、直升机以及机器人等领域；SG90舵机是分为两种的，一种是转角范围180° 的舵机，另外一种360° 转角的舵机。

180° 舵机可以通过20ms的PWM信号实现控制，可以通过调节PWM占空比调节转动角度。

360° 舵机可以通过20ms的PWM信号实现控制，可以通过调节PWM占空比调节转速。

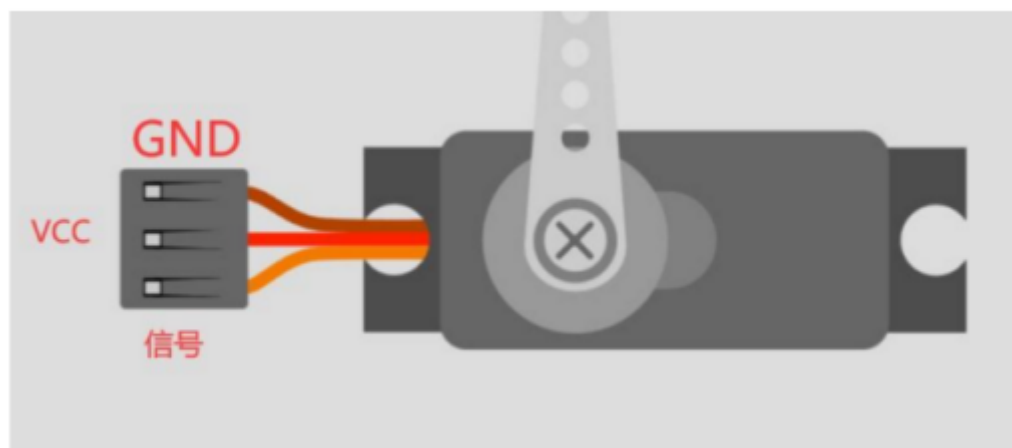
我们使用的是360° 舵机，电平持续时间与转速关系如下：

**360°舵机版本**不可控制角度，只能顺时针旋转、逆时针旋转、停止、调节转速。无角度定位，上电不会复位到 0°。因为这是 360°任意旋转的，没有 0°。通过一定参数的脉冲信号控制它的旋转，请看下表：

| 360度舵机控制参数 |                                                                                                                           |             |            |
|------------|---------------------------------------------------------------------------------------------------------------------------|-------------|------------|
| 舵机状态       | 脉冲周期                                                                                                                      | 脉冲高电平时间     | 对应占空比      |
| 正转         | 20ms                                                                                                                      | 0.5ms-1.5ms | 2.5%-7.5%  |
| 反转         | 20ms                                                                                                                      | 1.5ms-2.5ms | 7.5%-12.5% |
| 暂停         | 20ms                                                                                                                      | 1.5ms       | 7.50%      |
| 转速（）       | 20ms                                                                                                                      | 2.5ms       | 12.50%     |
| 注意：        | 由于每一个舵机的中位可能会不一样，所以有些舵机可能是1520us 的 PWM，舵机才会停下来。所以需要自己实际测试出舵机的中位。360度舵机就是一个普通的直流电机+一个电机驱动板的组合，所以它只能连续旋转，不能定位，也没法知道它的角度和圈数。 |             |            |

## 5.2 硬件连接

### 1. 舵机实物接线信息

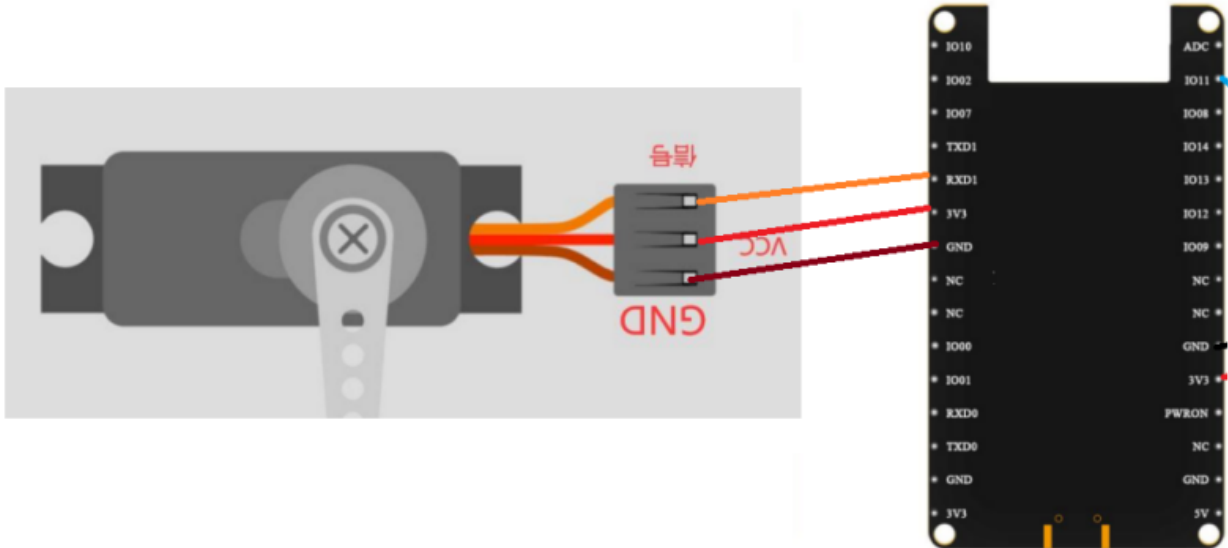


棕色 ----> GND

红色 ----> VCC

橙色 ---->信号线

### 2. 实物连接



信号线：所连接的GPIO引脚编号为GPIO5

编程说明：

PWM的输出可以使用时钟实现，也可以通过延时处理模拟输出PWM波。

SG90舵机一个周期是20ms，频率为  $1000\text{ms}/20\text{ms} = 50\text{hz}$

Hi3861开发板内部晶振是160Mhz，最大分频是65535，频率最小是2441 hz。

只能使用延时处理函数模拟PWM输出。

## 5.3 舵机编程实现

1. 完成工程的创建和配置
2. sg90舵机控制编程

```

1  #include <stdio.h>
2  #include <unistd.h>
3  #include "ohos_init.h"
4  #include "cmsis_os2.h"
5  #include "hi_time.h"
6  #include "hi_io.h"
7  #include "hi_gpio.h"
8
9  #define SG90_PIN          HI_IO_NAME_GPIO_5 // for hispark_pegasus
10 #define SG90_FUN_GPIO    HI_IO_FUNC_GPIO_2_GPIO
11
12 void SG90_Init(void)
13 {
14     /* gpio 初始化 */
15     hi_gpio_init();
16
17     /* GPIO引脚设置为通用GPIO引脚功能 */

```

```

18     hi_io_set_func(SG90_PIN, SG90_FUN_GPIO);
19
20     /* 设置GPIO管脚为输出 */
21     hi_gpio_set_dir(SG90_PIN, HI_GPIO_DIR_OUT);
22 }
23
24
25 int m_time=1000;    /* 调节m_time的值，可以调节舵机的转速和转动方向 */
26
27 void sg90_thread_func(void *arg)
28 {
29     SG90_Init();
30
31     while(1)
32     {
33         hi_gpio_set_output_val(SG90_PIN, HI_GPIO_VALUE1);    /* 引脚状态设置为高电平 */
34         hi_udelay(m_time); //延时1ms;
35
36         hi_gpio_set_output_val(SG90_PIN, HI_GPIO_VALUE0);    /* 引脚状态设置为低电平 */
37         hi_udelay(20*1000-m_time); //延时19ms;
38     }
39 }
40
41 static void ThreadSG90Task(void)
42 {
43     //定义线程属性
44     osThreadAttr_t attr;
45     attr.name = "sg90_thread_func";
46     attr.stack_size = 10*1024;
47     attr.priority = osPriorityNormal;
48     if (osThreadNew((osThreadFunc_t)sg90_thread_func, NULL, &attr) == NULL)
49     {
50         printf("[ThreadTestTask] Falied to create Task01!\n");
51     }
52 }
53 /* 声明函数入口 */
54 APP_FEATURE_INIT(ThreadSG90Task);

```

## 6 DHT11编程



## 6.1 DHT11概述

DHT11是单总线通信方式。

单总线通信与SPI、I2C等串行数据通信方式不同，因为它仅使用单根信号线进行数据传输。这种通信方式的数据传输是双向的，既能传输时钟信号，也能传输数据。单总线通信因其简单的结构、低成本以及易于维护等优点而被广泛应用于各种系统中，特别适用于单个主机系统。

DHT11数字温湿度传感器是一款含有已校准数字信号输出的温湿度复合传感器，它应用专用的数字模块采集技术和温湿度传感技术，确保产品具有极高的可靠性和卓越的长期稳定性。传感器包括一个电阻式感湿元件和一个NTC测温元件，并与一个高性能8位单片机相连接。因此该产品具有品质卓越、超快响应、抗干扰能力强、性价比极高等优点。每个DHT11传感器都在极为精确的湿度校验室中进行校准。校准系数以程序的形式存在OTP内存中，传感器内部在检测信号的处理过程中要调用这些校准系数。单线制串行接口，使系统集成变得简易快捷。超小的体积、极低的功耗，使其成为该类应用中，在苛刻应用场合的最佳选择。

## 6.2 DHT11通信数据帧编码格式

采用单总线双向串行通信协议，每次采集都要由单片机发起开始信号，然后DHT11会向单片机发送响应信号并开始传输40位数据帧，高位在前。数据格式为：8bit湿度整数数据+8bit湿度小数数据+8bit温度整数数据+8bit温度小数数据+8bit校验位，温湿度小数部分默认为0，即单片机采集的数据都是整数，校验位为4个字节的数据相加取结果的低8位数据作为校验和：

示例一：接收到的 40 位数据为：

|                  |                  |                  |                  |                  |
|------------------|------------------|------------------|------------------|------------------|
| <u>0011 0101</u> | <u>0000 0000</u> | <u>0001 1000</u> | <u>0000 0000</u> | <u>0100 1101</u> |
| 湿度高 8 位          | 湿度低 8 位          | 温度高 8 位          | 温度低 8 位          | 校验位              |

计算：

$$0011\ 0101 + 0000\ 0000 + 0001\ 1000 + 0000\ 0000 = 0100\ 1101$$

接收数据正确：

湿度：0011 0101=35H=53%RH

温度：0001 1000=18H=24℃

示例二：接收到的 40 位数据为：

|                  |                  |                  |                  |                  |
|------------------|------------------|------------------|------------------|------------------|
| <u>0011 0101</u> | <u>0000 0000</u> | <u>0001 1000</u> | <u>0000 0000</u> | <u>0100 1001</u> |
| 湿度高 8 位          | 湿度低 8 位          | 温度高 8 位          | 温度低 8 位          | 校验位              |

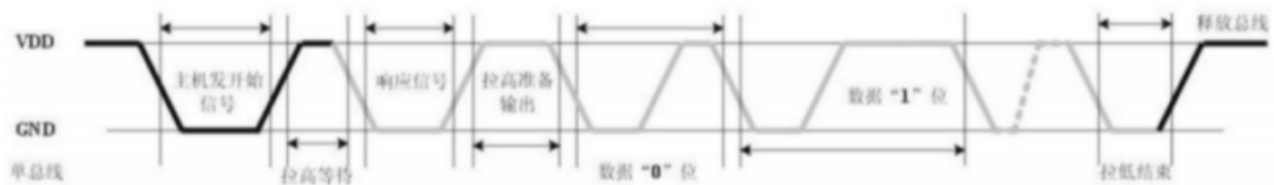
计算：

$$0011\ 0101 + 0000\ 0000 + 0001\ 1000 + 0000\ 0000 = 0100\ 1101$$

01001101 不等于 0100 1001

## 6.3 DHT11通信的时序

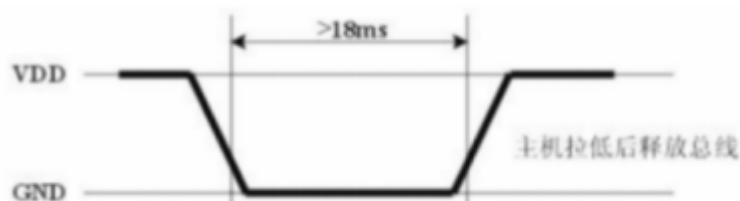
### 6.3.1 总时序



时序信号包含：

- 1) 开始信号                      主机发送，从机接收
- 2) 响应信号                    从机发送，主机接收
- 3) 数据传输信息（数据位1传输信号和数据位0传输信号）      从机发送，主机接收

### 6.3.2 开始信号



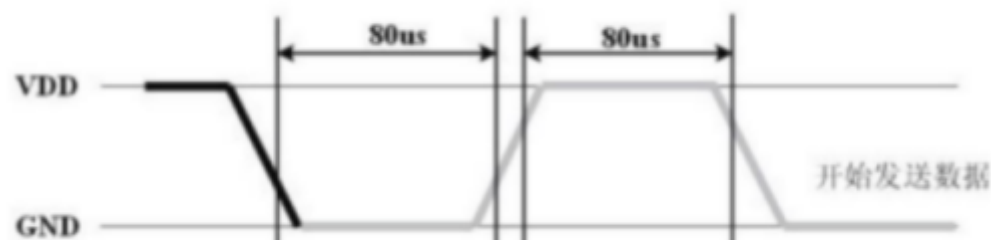
主机先将IO设置为输出，然后主机拉低总线（时间需要大于18ms）后设置为输入并拉高释放数据总线，等待从机（DHT11）响应

```

1 void DHT11_Rst(void)
2 {
3     DHT11_IO_OUT();           //将单总线引脚设置为输出模式（主机发送）
4     DHT11_DQ_OUT_Low;        //拉低DQ
5     hi_udelay(20000);         //拉低至少18ms
6     DHT11_DQ_OUT_High;       //DQ=1 释放单总线
7     hi_udelay(35);            //主机拉高20~40us,
8 }

```

### 6.3.3 响应信号



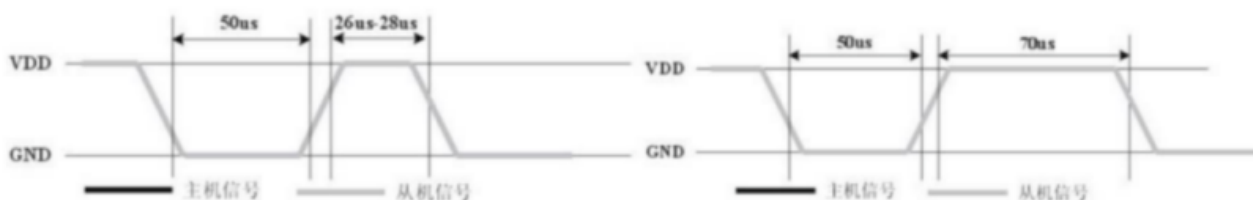
如果传感器正常且存在，则会在收到主机的开始信号后拉低总线并持续80us来通知主机此时传感器正常，然后拉高总线80us，通知主机准备接收；

```

1 //等待DHT11的回应
2 //返回1:未检测到DHT11的存在
3 //返回0:存在
4 u8 DHT11_Check(void)
5 {
6     u8 retry=0;
7     DHT11_IO_IN();//引脚设置为输入模式
8     while (GPIOGETINPUT(DHT11_GPIO,&DHT11_DQ_IN)&&retry<100)//DHT11会拉低40~80us
9     {
10         retry++;
11         hi_udelay(1);
12     };
13     if(retry>=100)return 1;
14     else retry=0;
15
16     while ((!GPIOGETINPUT(DHT11_GPIO,&DHT11_DQ_IN))&&retry<100)//DHT11拉低后会再次拉高
    40~80us
17     {
18         retry++;
19         hi_udelay(1);
20     };
21     if(retry>=100)return 1;
22     return 0;
23 }

```

### 6.3.4 数据位传输信号



数据位的传输分为**低电平信号**传输和**高电平信号**传输

时序说明:

- 1) 数据位传输的开始时序为50us的低电平信号;
- 2) 高电平信号的到达, 表示数据位传输的开始, 并且高电平信号根据传输的数据位值所持续不同长度的时间  
如果传输的数据位为高电平信号'1', 此时高电平信号持续时间约70us;  
如果传输的数据位为低电平信号'0', 此时高电平信号持续时间约为26us~28us;

数据位的接收:

- 1) 读取等待低电平信号的到达;

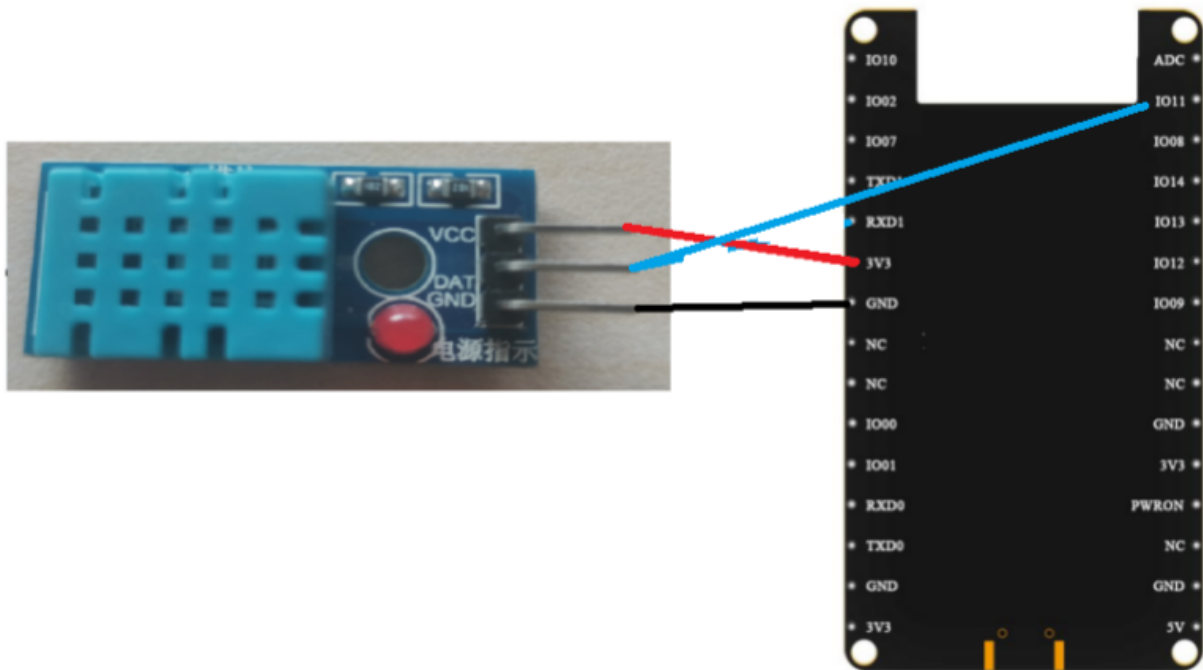
2) 循环读取电平信号直达高电平的到达。

3) 延时50us读取电平状态

```
1 //从DHT11读取一个位
2 //返回值: 1/0
3 u8 DHT11_Read_Bit(void)
4 {
5     u8 retry=0;
6     while(GPIOGETINPUT(DHT11_GPIO,&DHT11_DQ_IN)&&retry<100){//等待变为低电平
7         retry++;
8         hi_udelay(1);
9     }
10    retry=0;
11    while(!GPIOGETINPUT(DHT11_GPIO,&DHT11_DQ_IN)&&retry<100){//等待变高电平
12        retry++;
13        hi_udelay(1);
14    }
15    hi_udelay(40);//等待40us //用于判断高低电平, 即数据1或0
16    if(GPIOGETINPUT(DHT11_GPIO,&DHT11_DQ_IN))
17        return 1;
18    else
19        return 0;
20 }
```

## 6.4 DHT11实现

### 6.4.1 实物连接



DHT11实物模块中：

- 3.3v ----> 3v3
- GND ----> GND
- P0\_6 ----> IO11

## 6.4.2 DHT11编程实现

1. 创建工程目录
2. 在目录中添加DHT11驱动程序文件：m\_dht11.c 和 m\_dht11.h
  - 1) m\_dht11.h

```

1  #ifndef M_DHT11_H
2  #define M_DHT11_H

3  #include <stdio.h>
4  #include <unistd.h>

5  #include "ohos_init.h"

6  #include "cmsis_os2.h"
7  #include "hi_time.h"

8  #include "hi_io.h"
9  #include "hi_gpio.h"

10

11 typedef unsigned char u8;

```

```

12 #define DHT11_GPIO HI_IO_NAME_GPIO_11 //IO管脚定义
13 #define DHT11_GPIO_FUN HI_IO_FUNC_GPIO_2_GPIO //定义管脚功能函数

14 #define DHT11_DQ_OUT_High hi_gpio_set_ouput_val(DHT11_GPIO, 1); //设置GPIO输出高电平
15 #define DHT11_DQ_OUT_Low hi_gpio_set_ouput_val(DHT11_GPIO, 0); //设置GPIO输出低电平

16 typedef unsigned char u8;
17 extern u8 DHT11_Init(void);
18 extern u8 DHT11_Read_Data(u8 *temp,u8 *humi);

19 #endif

```

## 2) m\_dht11.c

```

1 #include "m_dht11.h"
2 hi_gpio_value DHT11_DQ_IN = {0};
3 /*****
4 设置端口为输出
5 *****/
6 void DHT11_IO_OUT(void)
7 {
8     //设置GPIO_11为输出模式
9     //GpioSetDir(DHT11_GPIO, WIFI_IOT_GPIO_DIR_OUT);
10    hi_gpio_init(); // GPIO初始化
11    hi_io_set_pull(DHT11_GPIO, HI_IO_PULL_UP); // 设置GPIO上拉
12    hi_io_set_func(DHT11_GPIO, DHT11_GPIO_FUN); // 设置IO为GPIO功能
13    hi_gpio_set_dir(DHT11_GPIO, HI_GPIO_DIR_OUT); // 设置GPIO为输出模式
14 }
15
16 //复位DHT11
17 void DHT11_Rst(void)
18 {
19     DHT11_IO_OUT(); //SET OUTPUT
20     DHT11_DQ_OUT_Low; //拉低DQ
21     hi_udelay(20000); //拉低至少18ms
22     DHT11_DQ_OUT_High; //DQ=1
23     hi_udelay(35); //主机拉高20~40us
24 }
25
26

```

```

27 //获取GPIO输入状态
28 u8 GPIOGETINPUT(hi_gpio_idx id,hi_gpio_value *val)
29 {
30     hi_gpio_get_input_val(id,val);
31     return *val;
32 }
33
34 /*****
35 设置端口为输入
36 *****/
37 void DHT11_IO_IN(void)
38 {
39     hi_gpio_set_dir(DHT11_GPIO, HI_GPIO_DIR_IN);           // 设置GPIO为输入模式
40     hi_io_set_pull(DHT11_GPIO, HI_IO_PULL_NONE);           // 设置GPIO浮空
41 }
42
43
44
45 //等待DHT11的回应
46 //返回1:未检测到DHT11的存在
47 //返回0:存在
48 u8 DHT11_Check(void)
49 {
50     u8 retry=0;
51     DHT11_IO_IN();//SET INPUT
52     while (GPIOGETINPUT(DHT11_GPIO,&DHT11_DQ_IN)&&retry<100)//DHT11会拉低40~80us
53     {
54         retry++;
55         hi_udelay(1);
56     };
57     if(retry>=100)return 1;
58     else retry=0;
59
60     while ((!GPIOGETINPUT(DHT11_GPIO,&DHT11_DQ_IN))&&retry<100)//DHT11拉低后会再次拉高
61     40~80us
62     {
63         retry++;
64         hi_udelay(1);
65     };
66     if(retry>=100)return 1;
67     return 0;
68 }

```

```

68
69 //初始化DHT11的IO口 DQ 同时检测DHT11的存在
70 //返回1:不存在
71 //返回0:存在
72 u8 DHT11_Init(void)
73 {
74     //初始化GPIO
75     hi_gpio_init(); // GPIO初始化
76     //设置GPIO_11的复用功能为普通GPIO
77     hi_io_set_func(DHT11_GPIO, DHT11_GPIO_FUN);
78     //设置GPIO_11为输出模式
79     hi_gpio_set_dir(DHT11_GPIO, HI_GPIO_DIR_OUT);
80     //设置GPIO_11输出高电平
81     //GpioSetOutputVal(DHT11_GPIO, 1);
82     hi_io_set_pull(DHT11_GPIO, HI_IO_PULL_UP); // 设置GPIO上拉
83     DHT11_Rst(); //复位DHT11
84     return DHT11_Check();//等待DHT11的回应
85 }
86
87 //从DHT11读取一个位
88 //返回值: 1/0
89 u8 DHT11_Read_Bit(void)
90 {
91     u8 retry=0;
92     while(GPIOGETINPUT(DHT11_GPIO,&DHT11_DQ_IN)&&retry<100){//等待变为低电平
93         retry++;
94         hi_udelay(1);
95     }
96     retry=0;
97     while((!GPIOGETINPUT(DHT11_GPIO,&DHT11_DQ_IN))&&retry<100){//等待变高电平
98         retry++;
99         hi_udelay(1);
100     }
101     hi_udelay(40);//等待40us //用于判断高低电平, 即数据1或0
102     if(GPIOGETINPUT(DHT11_GPIO,&DHT11_DQ_IN))return 1; else return 0;
103 }
104 //从DHT11读取一个字节
105 //返回值: 读到的数据
106 u8 DHT11_Read_Byte(void)
107 {
108     u8 i,dat;
109     dat=0;

```



```

110     for (i=0;i<8;i++)
111     {
112         dat<<=1;
113         dat|=DHT11_Read_Bit();
114     }
115     return dat;
116 }
117 //从DHT11读取一次数据
118 //temp:温度值(范围:0~50°)
119 //humi:湿度值(范围:20%~90%)
120 //返回值: 0,正常;1,读取失败
121 u8 DHT11_Read_Data(u8 *temp,u8 *humi)
122 {
123     u8 buf[5]={ 0 };
124     u8 i;
125     DHT11_Rst();
126     if(DHT11_Check()==0)
127     {
128         for(i=0;i<5;i++)//读取40位数据
129         {
130             buf[i]=DHT11_Read_Byte();
131         }
132         if((buf[0]+buf[1]+buf[2]+buf[3])==buf[4])//数据校验
133         {
134             *humi=buf[0];
135             *temp=buf[2];
136         }
137     }else return 1;
138     return 0;
139 }

```

### 3. 创建应用程序文件并编程: dht11\_demo.c

```

1  #include <stdio.h>
2  #include <unistd.h>
3  #include "ohos_init.h"
4  #include "cmsis_os2.h"
5  #include "hi_time.h"
6  #include "hi_io.h"
7  #include "hi_gpio.h"

```

```

8  #include "m_dht11.h"
9
10
11 void dht11_thread_func(void *arg)
12 {
13     u8 temp;
14     u8 humi;
15
16     DHT11_Init();
17
18     while(1)
19     {
20         if (0 == DHT11_Read_Data(&temp, &humi)) {
21             printf("dht11:temp = %d, humi = %d\n", temp, humi);
22             osDelay(200); //延时2s
23         }
24     }
25 }
26
27 static void ThreadDHT11Task(void)
28 {
29     //定义线程属性
30     osThreadAttr_t attr;
31     attr.name = "dht11_thread_func";
32     attr.stack_size = 10*1024;
33     attr.priority = osPriorityNormal;
34     if (osThreadNew((osThreadFunc_t)dht11_thread_func, NULL, &attr) == NULL)
35     {
36         printf("[ThreadTestTask] Falied to create Task01!\n");
37     }
38 }
39 /* 声明函数入口 */
40 APP_FEATURE_INIT(ThreadDHT11Task);

```

## 知识点4 网络的通信

### 1 WiFi的连接

